

Unit – I : Operating System Concepts

1.1 Need of Operating System

An **Operating System (OS)** is a collection of system programs that **controls the overall operation of a computer system**. It acts as an **interface between the user and computer hardware**, making the system easy to use and efficient.

Why Operating System is Needed

1. **Interface Between User and Hardware**
 - Users cannot directly interact with hardware.
 - OS provides interfaces such as **GUI (Graphical User Interface)** and **CLI (Command Line Interface)**.
2. **Efficient Resource Utilization**
 - Manages CPU, memory, input/output devices, and storage.
 - Prevents conflicts when multiple programs request the same resource.
3. **Process Management**
 - Creates, schedules, executes, and terminates processes.
 - Enables multitasking and multiprocessing.
4. **Memory Management**
 - Allocates memory to programs when required.
 - Deallocates memory after program completion.
 - Ensures no two programs access the same memory improperly.
5. **File Management**
 - Organizes data into files and directories.
 - Controls access permissions and storage allocation.
6. **Security and Protection**
 - Protects system resources from unauthorized access.
 - Uses passwords, authentication, and access control mechanisms.
7. **Error Detection and Handling**
 - Detects hardware and software errors.
 - Takes corrective actions to maintain system stability.

1.2 Structure of Operating System

The **structure of an operating system** defines **how its components are arranged and how they interact with each other**. A good structure improves **performance, reliability, and maintainability**.

1.2.1 System Components

The major components of an operating system are:

1. Process Management

- A **process** is a program in execution.
- OS functions:
 - Process creation and termination
 - CPU scheduling
 - Process synchronization
 - Deadlock handling

2. Memory Management

- Controls main memory usage.
- Responsibilities:
 - Tracking memory usage
 - Memory allocation and deallocation
 - Virtual memory management

3. File System Management

- Organizes and manages files on storage devices.
- Functions:
 - Create, delete, read, and write files
 - Directory management
 - File protection

4. I/O Device Management

- Manages input/output devices like keyboard, mouse, printer.
- Uses **device drivers** to communicate with hardware.

5. Secondary Storage Management

- Handles hard disks and SSDs.
- Includes:
 - Disk scheduling
 - Free space management
 - Storage allocation

6. Protection and Security

- Ensures controlled access to resources.
- Prevents unauthorized use of data and system programs.

1.2.2 System Calls

System calls are the **programming interface between user programs and the operating system**. They allow programs to request services from the OS.

Functions of System Calls

- Provide controlled access to hardware
- Enable interaction between user programs and OS kernel

Types of System Calls

1. **Process Control**
 - Create process
 - End process
 - Load and execute program
 2. **File Management**
 - Create, open, read, write, close files
 - Delete files
 3. **Device Management**
 - Request device
 - Release device
 - Read/write device
 4. **Information Maintenance**
 - Get system time
 - Get system data
 - Set system attributes
 5. **Communication**
 - Message passing
 - Shared memory
-

1.2.3 System Structure

1. Simple Structure

- No proper division of components
- Fast but difficult to maintain
- Example: **MS-DOS**

2. Monolithic Structure

- All OS services run in a single kernel space
- High performance but less secure
- Example: **UNIX**

3. Layered Structure

- OS divided into layers
- Each layer performs a specific function

- Easy to debug and maintain

4. Microkernel Structure

- Only essential services in kernel
 - Other services run in user space
 - More reliable and secure
-

1.3 Operating System Types

Operating systems are classified based on **how they manage processes and resources**.

1.3.1 Batch Operating System

- Jobs are collected and processed in batches.
- No user interaction during execution.

Advantages

- High throughput

Disadvantages

- Long waiting time
 - No immediate feedback
-

1.3.2 Multiprogramming Operating System

- Multiple programs stored in memory.
- CPU switches among programs to increase utilization.

Advantages

- Efficient CPU usage

Disadvantages

- Complex memory management
-

1.3.3 Multitasking Operating System

- Multiple tasks executed simultaneously by rapid switching.
- Improves system responsiveness.

Examples

- Windows, Linux, macOS
-

1.3.4 Multiuser Operating System

- Supports multiple users at the same time.
- Each user gets a separate environment.

Examples

- UNIX, Linux servers
-

1.3.5 Time Sharing Operating System

- CPU time is divided into small time slices.
- Each user gets equal time.

Advantages

- Fast response
 - Interactive environment
-

1.3.6 Real-Time Operating System (RTOS)

- Provides immediate response within fixed deadlines.
- Used in critical systems.

Types of RTOS

1. **Hard RTOS**
 - Strict deadlines
 - Example: Aircraft control systems
 2. **Soft RTOS**
 - Deadlines are flexible
 - Example: Multimedia systems
-

1.4 Operating System Services

Operating systems provide various services to users and programs.

Major OS Services

1. **User Interface**
 - CLI or GUI
2. **Program Execution**
 - Loads and runs programs
3. **I/O Operations**
 - Manages input and output devices
4. **File System Operations**
 - Create, read, write, delete files
5. **Communication**
 - Enables inter-process communication
6. **Error Detection**
 - Detects hardware and software errors
7. **Resource Allocation**
 - Allocates CPU, memory, and devices
8. **Accounting**
 - Keeps track of resource usage
9. **Protection and Security**
 - Ensures controlled access to system resources

Unit – II : Process & CPU Management

2.1 Process Concept

A **process** is a **program in execution**. When a program is loaded into memory and starts running, it becomes a process. A process includes:

- Program code
- Data
- CPU registers
- Program counter
- Stack and heap

A computer system may have **multiple processes running simultaneously**, managed by the operating system.

2.1.1 Programmer's View of Process

From a **programmer's point of view**, a process is:

- An active entity
- A running instance of a program
- Having its own execution flow

Program vs Process

Program	Process
Passive entity	Active entity
Stored on disk	Resides in memory
Static	Dynamic
One program can create many processes	Each process is unique

Example:

One text editor program can open multiple documents → each document is a separate **process**.

2.1.2 Process State

During its lifetime, a process passes through several **states**.

Basic Process States

1. **New**
 - Process is being created
2. **Ready**
 - Process is loaded in memory and waiting for CPU
3. **Running**
 - Process is currently being executed by CPU
4. **Waiting / Blocked**
 - Process is waiting for an event (I/O, user input)
5. **Terminated**
 - Process has finished execution

State Transitions

- Ready → Running (CPU allocated)
 - Running → Waiting (I/O request)
 - Running → Ready (Time slice expired)
 - Running → Terminated (Execution completed)
-

2.1.3 Process Control Block (PCB)

A **Process Control Block (PCB)** is a **data structure used by the operating system** to store all information about a process.

Contents of PCB

1. **Process ID (PID)**
 - Unique identification number
2. **Process State**
 - New, Ready, Running, Waiting, Terminated
3. **Program Counter**
 - Address of next instruction
4. **CPU Registers**
 - Register values used by process
5. **CPU Scheduling Information**
 - Priority, scheduling queue pointers
6. **Memory Management Information**
 - Base and limit registers
7. **I/O Status Information**
 - List of allocated I/O devices

PCB allows the OS to **perform context switching** efficiently.

2.1.4 Process Scheduling

Process scheduling is the activity of selecting one process from the ready queue and allocating the CPU to it.

Objectives of Process Scheduling

- Maximize CPU utilization
- Minimize waiting time
- Ensure fairness
- Improve system performance

Scheduling Queues

- **Job Queue** – all processes in system
 - **Ready Queue** – processes ready for CPU
 - **Device Queue** – processes waiting for I/O
-

2.2 CPU Scheduling

CPU scheduling decides **which process will get CPU and for how long**.

When CPU becomes idle, OS selects a process from the ready queue.

2.2.1 Schedulers

Schedulers are system programs that select processes for execution.

Types of Schedulers

1. **Long-Term Scheduler**
 - Selects processes from job pool
 - Controls degree of multiprogramming
 2. **Short-Term Scheduler**
 - Selects one process from ready queue
 - Executes frequently (very fast)
 3. **Medium-Term Scheduler**
 - Performs swapping (suspend/resume processes)
-

2.2.2 Scheduling Criteria

CPU scheduling algorithms are evaluated using the following criteria:

1. **CPU Utilization**

- Keep CPU as busy as possible
 - 2. **Throughput**
 - Number of processes completed per unit time
 - 3. **Turnaround Time**
 - Time from submission to completion
 - 4. **Waiting Time**
 - Time spent waiting in ready queue
 - 5. **Response Time**
 - Time between request and first response
-

2.2.3 Scheduling Algorithms

1. First Come First Serve (FCFS)

- Processes executed in order of arrival
- Simple but may cause **convoy effect**

Advantages

- Easy to implement

Disadvantages

- High waiting time
-

2. Shortest Job First (SJF)

- Process with smallest execution time runs first
- Optimal for minimum waiting time

Disadvantages

- Difficult to predict CPU burst time
 - May cause starvation
-

3. Priority Scheduling

- Process with highest priority executes first
- Can be preemptive or non-preemptive

Problem

- Starvation of low-priority processes
-

4. Round Robin (RR)

- Each process gets fixed time quantum
- Best for time-sharing systems

Advantages

- Fair
 - Good response time
-

2.3 Deadlock

A **deadlock** is a situation where **two or more processes are waiting indefinitely for resources held by each other.**

2.3.1 Basic Concepts

Necessary Conditions for Deadlock

(All four must occur simultaneously)

1. **Mutual Exclusion**
 - Resource cannot be shared
 2. **Hold and Wait**
 - Process holds one resource and waits for another
 3. **No Preemption**
 - Resources cannot be forcibly taken
 4. **Circular Wait**
 - Circular chain of processes waiting for resources
-

2.3.2 Deadlock Detection

- OS periodically checks for deadlocks
- Uses **resource allocation graph**
- If a cycle exists → deadlock detected

Disadvantage

- Detection overhead
-

2.3.3 Deadlock Prevention

Deadlock prevention ensures **at least one necessary condition never holds**.

Methods

- Remove mutual exclusion (not always possible)
 - Avoid hold and wait
 - Allow preemption
 - Break circular wait
-

2.3.4 Deadlock Avoidance

Deadlock avoidance requires **advance information** about resource requirements.

Banker's Algorithm

- Most common deadlock avoidance algorithm
- System checks for **safe state** before resource allocation

Safe State

- System can allocate resources without deadlock

Unit – III : Memory Management

3.1 Need of Memory Management

Memory management is one of the most important functions of an operating system. It is responsible for **efficient utilization of primary memory (RAM)** among multiple processes.

Why Memory Management is Needed

1. **Efficient Use of Memory**
 - RAM is limited and expensive.
 - OS must allocate memory carefully to avoid wastage.
 2. **Process Isolation**
 - Each process should have its own memory space.
 - Prevents one process from accessing another process's memory.
 3. **Multiprogramming Support**
 - Multiple processes reside in memory at the same time.
 - OS decides how much memory each process gets.
 4. **Protection and Security**
 - Protects programs and OS from illegal memory access.
 5. **Virtual Memory Support**
 - Allows execution of programs larger than physical memory.
 6. **Performance Improvement**
 - Proper memory allocation improves CPU utilization.
-

3.2 Contiguous Allocation

In **contiguous memory allocation**, each process is allocated **one continuous block of memory**.

Characteristics

- Simple to implement
 - Memory allocated in a single block
 - May cause fragmentation
-

3.2.1 Partitioned Memory Allocation

Memory is divided into **partitions**, and each partition is allocated to one process.

Types of Partitioning

- Fixed partitioning
 - Variable partitioning
-

3.2.2 Fixed & Variable Partitioning

Fixed Partitioning

- Memory is divided into fixed-size partitions.
- Number of partitions is decided in advance.

Advantages

- Simple memory management

Disadvantages

- **Internal fragmentation**
 - Limited number of processes
-

Variable Partitioning

- Memory partitions are created dynamically as per process size.
- Partition size varies.

Advantages

- Better memory utilization
- No internal fragmentation

Disadvantages

- **External fragmentation**
 - Complex memory management
-

3.2.3 Swapping

Swapping is a technique in which processes are **temporarily moved from main memory to secondary storage (disk)** to free memory.

Working of Swapping

- OS swaps out a process to disk

- Brings another process into memory
- Swapped-in process resumes execution

Advantages

- Supports multiprogramming
- Better memory utilization

Disadvantages

- High disk I/O overhead
 - Slow performance
-

3.2.4 Relocation

Relocation allows a process to be **moved from one memory location to another** during execution.

Types of Relocation

1. **Static Relocation**
 - Done at compile or load time
2. **Dynamic Relocation**
 - Done during execution using relocation registers

Advantages

- Efficient memory utilization
 - Supports swapping
-

3.2.5 Protection and Sharing

Memory Protection

- Prevents unauthorized access to memory.
- Uses **base and limit registers**.

Memory Sharing

- Allows multiple processes to share common memory.
- Example: Shared libraries

Benefits

- Saves memory
 - Faster execution
-

3.3 Non-Contiguous Allocation

In **non-contiguous memory allocation**, a process is divided into parts and stored in **different memory locations**.

Advantages

- Eliminates external fragmentation
 - Efficient memory utilization
-

3.3.1 Paging

Paging divides:

- Physical memory into **frames**
- Logical memory into **pages**

Paging Process

- Page size = Frame size
- Page table maps pages to frames

Advantages

- No external fragmentation
- Supports virtual memory

Disadvantages

- Page table overhead
 - Internal fragmentation possible
-

3.3.2 Segmentation

Segmentation divides a process into **logical segments** such as:

- Code segment
- Data segment
- Stack segment

Segment Table

- Segment number
- Base address
- Segment limit

Advantages

- Logical view of memory
- Supports protection and sharing

Disadvantages

- External fragmentation
 - Complex memory management
-

3.4 Virtual Memory

Virtual memory is a memory management technique that allows execution of programs **larger than physical memory**.

Advantages

- Efficient memory utilization
 - More programs can run simultaneously
 - Large address space
-

3.4.1 Demand Paging

In **demand paging**, pages are loaded into memory **only when required**.

Page Fault

- Occurs when required page is not in memory
- OS loads page from disk

Advantages

- Reduced memory usage
 - Faster program start
-

3.4.2 Page Replacement Algorithms

When memory is full and a page fault occurs, OS replaces a page using replacement algorithms.

Common Page Replacement Algorithms

1. **FIFO (First In First Out)**
 - Oldest page is replaced
 - Simple but inefficient
 2. **LRU (Least Recently Used)**
 - Replaces least recently used page
 - Good performance but complex
 3. **Optimal Algorithm**
 - Replaces page that will not be used for longest time
 - Best performance (theoretical)
-

3.4.3 Thrashing

Thrashing occurs when the system spends **more time swapping pages than executing processes**.

Causes of Thrashing

- High degree of multiprogramming
- Insufficient memory

Effects

- Low CPU utilization
- Poor system performance

Prevention

- Reduce multiprogramming level
 - Use efficient page replacement algorithms
-

Unit – IV : File Management

4.1 File Management

File management is the part of the operating system that **handles creation, storage, organization, access, and protection of files** stored on secondary storage devices such as hard disks and SSDs.

A **file** is a collection of related information stored on a storage device and identified by a name.

Objectives of File Management

- Efficient storage and retrieval of data
 - Easy access to files
 - Protection and security of data
 - Sharing of files among users and processes
-

4.1.1 File Concept and Access Methods

File Concept

A file consists of:

- **Name** – human-readable identifier
- **Data** – actual contents of file
- **Metadata** – information about the file

Files are stored in **secondary memory** and managed by the file system.

File Access Methods

File access methods define **how data is read from or written to a file**.

1. Sequential Access

- Data accessed in order, one after another
- Example: Text files, audio/video streams

Advantages

- Simple to implement

Disadvantages

- Slow for random data access
-

2. Direct (Random) Access

- File can be accessed at any position
- Example: Database files

Advantages

- Fast access to records
-

3. Indexed Access

- Index contains pointers to different blocks of file
 - Combines advantages of sequential and direct access
-

4.1.2 User View of File System

From the **user's perspective**, the file system provides:

1. **Hierarchical Structure**
 - Files organized into directories and subdirectories
2. **Naming**
 - Each file has a unique name within a directory
3. **File Operations**
 - Create, delete, open, close, read, write
4. **Protection**
 - Access permissions for users

Users do not need to know **physical storage details**; OS handles it internally.

4.1.3 File Attributes and Operations

File Attributes

File attributes provide information about a file.

Common attributes include:

- **Name**
 - **Type**
 - **Size**
 - **Location**
 - **Protection**
 - **Creation, modification, and access time**
 - **Owner/User ID**
-

File Operations

Common file operations supported by OS:

1. Create a file
 2. Open a file
 3. Read from a file
 4. Write to a file
 5. Close a file
 6. Delete a file
 7. Rename a file
-

4.1.4 File System Design

File system design focuses on:

- How files are stored
- How fast they can be accessed
- How reliably data is maintained

Design Goals

- High performance
- Reliability
- Scalability
- Security

Key Components

- File control blocks (FCB)
 - Directory structures
 - Allocation methods
 - Free space management
-

4.1.5 Disk Space Management

Disk space management ensures **efficient use of storage space**.

Free Space Management Techniques

1. **Bit Vector**
 - Each bit represents a block (free or occupied)
 2. **Linked List**
 - Free blocks linked together
 3. **Grouping**
 - Stores addresses of free blocks in groups
 4. **Counting**
 - Keeps count of consecutive free blocks
-

4.2 Directory Structure

A **directory** is a special file that contains information about files and subdirectories.

Types of Directory Structures

1. **Single-Level Directory**
 - All files in one directory
 - Simple but name conflicts occur
 2. **Two-Level Directory**
 - Separate directory for each user
 3. **Tree-Structured Directory**
 - Hierarchical structure
 - Most commonly used
 4. **Acyclic Graph Directory**
 - Allows sharing of files
 5. **General Graph Directory**
 - Supports cycles (complex to manage)
-

4.3 Disk Organization

Disk organization defines **how data is physically and logically stored on disks**.

4.3.1 Physical Structure

A disk is divided into:

- **Platter**
- **Track**
- **Sector**
- **Cylinder**

Characteristics

- Data stored in sectors
 - Access time depends on head movement and rotation
-

4.3.2 Logical Structure

Logical structure refers to **how OS views disk data**.

- Disk divided into **blocks**
 - Files stored as a collection of blocks
 - Logical view is independent of physical disk layout
-

4.3.3 Addressing

Disk addressing specifies **how data blocks are identified**.

Types of Addressing

- **Block Addressing**
 - **Cylinder-Head-Sector (CHS)**
 - **Logical Block Addressing (LBA)**
-

4.3.4 Disk Scheduling

Disk scheduling decides **the order in which disk I/O requests are serviced** to minimize seek time.

Disk Scheduling Algorithms

1. **FCFS (First Come First Serve)**
 - Simple but inefficient
2. **SSTF (Shortest Seek Time First)**
 - Selects closest request
3. **SCAN (Elevator Algorithm)**
 - Head moves in one direction servicing requests
4. **C-SCAN**

- Circular SCAN, uniform waiting time
-

4.4 Security and Protection Mechanism

Security and protection mechanisms ensure **safe and authorized access to files and system resources**.

Protection Mechanisms

1. **Access Control**
 - Read, write, execute permissions
 2. **User Authentication**
 - Passwords, user IDs
 3. **File Protection**
 - Owner, group, and others permissions
 4. **Encryption**
 - Protects data from unauthorized reading
 5. **Backup and Recovery**
 - Prevents data loss
-

Unit – V : Linux Basics

5.1 Overview of Linux

Linux is a **free and open-source operating system** based on UNIX. It was developed by **Linus Torvalds** in 1991. Linux is widely used in servers, desktops, mobile devices, and embedded systems.

Features of Linux

- Open source and free
- Multiuser operating system
- Multitasking operating system
- High security
- Portable and scalable
- Supports networking
- Strong command-line interface

Components of Linux

1. **Kernel** – Core of Linux OS
 2. **Shell** – Command interpreter
 3. **File System**
 4. **System Utilities**
 5. **Application Programs**
-

5.2 Installation and Upgrade

Linux Installation

Linux can be installed:

- As a **standalone OS**
- Along with Windows (**dual boot**)
- On **virtual machines**

Basic Installation Steps

1. Boot from installation media (USB/DVD)
2. Select language and keyboard
3. Disk partitioning
4. Create user account and password
5. Install packages

6. Restart system

Linux Upgrade

- Updates existing Linux system
 - Preserves user data and configurations
 - Done using package managers (apt, yum)
-

5.3 System Administration

System administration involves managing and maintaining Linux systems.

System Administrator Responsibilities

- User management
- File and directory management
- Software installation and updates
- System monitoring
- Backup and recovery
- Security management

Common Administrative Tasks

- Add/delete users
 - Change file permissions
 - Monitor processes
 - Manage disk space
-

5.4 Introduction to Shell and Commands

Shell

The **shell** is a program that **accepts commands from users and executes them**.

Common Linux Commands

User and System Information

- `who` – Displays logged-in users
 - `who am i` – Shows current user
 - `tty` – Displays terminal name
-

Directory Commands

- `pwd` – Shows present working directory
 - `cd` – Changes directory
 - `mkdir` – Creates directory
 - `rmdir` – Removes empty directory
 - `ls` – Lists files and directories
-

File Commands

- `cat` – Displays file content
 - `cp` – Copies files
 - `rm` – Removes files
 - `mv` – Moves or renames files
-

Text Processing Commands

- `wc` – Counts lines, words, characters
 - `split` – Splits large files
 - `cmp` – Compares two files
 - `comm` – Compares sorted files
 - `diff` – Shows differences
 - `head` – Displays first lines
 - `tail` – Displays last lines
 - `grep` – Searches text pattern
 - `sort` – Sorts file content
-

File Linking and Permissions

- `ln` – Creates links
 - `chmod` – Changes file permissions
 - `chown` – Changes owner
 - `chgrp` – Changes group
-

Printing and Expression

- `pr` – Formats files for printing
 - `lp` – Sends files to printer
 - `expr` – Evaluates expressions
-

Help and Manual

- `man` – Displays manual pages
 - `help` – Displays help for shell commands
-

Pipes and I/O Redirection

- `|` – Pipe (output of one command to another)
 - `>` – Output redirection
 - `<` – Input redirection
 - `>>` – Append output
 - `tee` – Displays output and saves to file
-

Process Command

- `ps` – Displays running processes
-

5.5 Edit Files with “vi” Editor

vi is a powerful text editor in Linux.

Modes of vi Editor

1. **Command Mode**
 - Default mode
 - Used for navigation and commands
2. **Insert Mode**
 - Used to insert text
 - Press `i`, `a`, or `o`
3. **Last Line Mode**
 - Used for saving, exiting
 - Press `:`

Common vi Commands

- `:w` – Save file
 - `:q` – Quit
 - `:wq` – Save and quit
 - `:q!` – Quit without saving
-

5.6 Linux

5.6.1 Shell Concept

A **shell** is an interface between the **user and the Linux kernel**.

Functions of Shell

- Command execution
- Variable handling
- Script execution
- I/O redirection

Types of Shell

- Bourne Shell (sh)
 - Bash Shell (bash)
 - C Shell (csh)
 - Korn Shell (ksh)
-

5.6.2 Basic Shell Scripts

A **shell script** is a file containing **a series of Linux commands** executed sequentially.

Advantages of Shell Scripts

- Automation
- Saves time
- Reduces errors

Simple Shell Script Example

```
#!/bin/bash
echo "Hello Linux"
date
```

Steps to Execute Script

1. Create file: `vi script.sh`
2. Give permission: `chmod +x script.sh`
3. Run: `./script.sh`